

00 TODO Airflow 설치 테스트

- TODO
- airflow helm 설치 (사내 docker-성수센터)
 - 설치
 - 접근
 - 설치 삭제
 - dags, logs 디렉토리 연결
 - Custom pod image 적용
 - Dag 수행
 - 로컬환경 task 수행 방법
 - 참고 - Custom airflow image 생성, registry 등록
 - 참고 - pv 경로 변경시 적용 방법
 - 참고 - 성수 k8s airflow → AWS OpenSearch 연동
 - 기타 참고
- airflow aws ec2 설치

TODO

- logs 생성 권한 - uid=50000 gid=0 으로 파일 생성됨
 - ls -al /nfsdata/data-airflow-logs
- ```
drwxrwxrwx 4 puser puser 4096 9월 1 10:16 .
drwxr-xr-x 3 puser puser 4096 9월 1 10:12 ..
drwxrwxr-x 3 50000 root 4096 9월 1 10:16 'dag_id=tutorial_timesleep'
drwxrwxr-x 3 50000 root 4096 9월 1 10:15 dag_processor
```
- 03-my-airflow-pod/Dockerfile 정의 내용
    - ARG UID=50000
    - ARG GID=50000
- custom-values.yaml.org → custom-values.yaml 적용 내용 재확인
  - airflow user create (참조 - <https://code.skplanet.com/projects/SEAR/repos/search-iac-airflow-v3/browse>)
  - airflow run dag 방법 - 1) UI 접근 수행 방법 2) CLI 명령 수행 방법은? 아래 "로컬환경 task 수행 방법" 참조

## airflow helm 설치 (사내 docker-성수센터)

test-airflow repo 참조

### 설치

- helm 설치
  - helm upgrade --install airflow apache-airflow/airflow --namespace airflow --create-namespace --debug --values custom-values.yaml (성공-postgresql 기동 성공)
- 실패 원인 - postgresql 기동 실패 - default storageclass가 없음. pvc가 자동 생성되지 않음

```
% k get pvc -A
NAMESPACE NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS AGE
airflow data-airflow-postgresql-0 Bound postgres-pv-volume 5Gi RWO manual <unset> 5m30s
airflow logs-airflow-triggerer-0 Pending local-storage <unset> 46h
airflow logs-airflow-worker-0 Pending local-storage <unset> 46h
airflow redis-db-airflow-redis-0 Pending
```

- 해결 방법 - pv, pvc 수동 생성
  - postgresql pv 생성시 nfs volume을 이용하도록 함 (다른 worker에서 떠도 정상 기동하도록) - nfs volume 미이용시 migration pod 수행 실패함

```

1) 기존 pvc 삭제
k -n airflow delete statefulsets airflow-postgresql (먼저 삭제해야 pvc 삭제 가능)
k -n airflow delete pvc -n airflow data-airflow-postgresql-0

2) pv, pvc 수동 생성
% cat pv01-postgresql.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
 name: postgres-pv
 labels:
 type: local
spec:
 storageClassName: manual
 capacity:
 storage: 5Gi
 accessModes:
 - ReadWriteOnce
 hostPath:
 path: "/tmp/data1"
--- # <- 이거 꼭 필요
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 namespace: airflow
 name: data-airflow-postgresql-0
spec:
 storageClassName: manual
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 5Gi
% k apply -f postgre-pv.yaml

3) postgresql 재기동
% helm upgrade --install airflow apache-airflow/airflow --namespace airflow --create-namespace --debug --values custom-values.yaml

4) pods 확인
% k -n airflow get pods
NAME READY STATUS RESTARTS AGE
airflow-api-server-6d6dbcbbfb-m5tjx 1/1 Running 0 12m
airflow-dag-processor-6774fbb9d7-v7c58 2/2 Running 0 45h
airflow-postgresql-0 1/1 Running 0 12m
airflow-redis-0 0/1 Pending 0 45h
airflow-scheduler-67656f96db-gszp4 2/2 Running 0 12m
airflow-statsd-6d69b6d684-b9gtg 1/1 Running 0 45h
airflow-triggerer-0 0/2 Pending 0 45h
airflow-worker-0 0/2 Pending 0 45h

```

- Pending 상태 나머지도 비슷하게 적용 - redis, trigger, worker

## 접근

- kubectl port-forward svc/airflow-api-server 8080:8080 --namespace airflow (default pw는 admin)

## 설치 삭제

- helm uninstall airflow --namespace airflow

## dags, logs 디렉토리 연결

- 참조 - <https://airflow.apache.org/docs/helm-chart/stable/airflow-configuration.html>
- 1) worker 서버 /nfsdata mount
  - 개발환경 서버포털에서 NAS 볼륨 생성

- worker 서버 접속후 LDAP 계정에서 root 전환
  - sudo su - (pw는 LDAP pw)
- mount 명령 수행
  - mount -o nolock 172.21.104.249:/THYKtx\_worker\_dev01\_n01\_287326 /nfsdata
- dags 파일 복사후 파일 소유권 모두 755로 변경 (puser 계정에서 작업하므로 airflow 계정으로 파일을 읽을 수 있도록 함)
- 2) pv, pvc 생성
- 3) custom-values.yaml 작성

```
dags:
 persistence:
 enabled: true
 existingClaim: airflow-dags
 gitSync:
 enabled: false
logs:
 persistence:
 enabled: true
 existingClaim: airflow-logs
```

- 4) airflow upgrade
  - helm upgrade airflow apache-airflow/airflow --namespace airflow --debug --values custom-values.yaml

## Custom pod image 적용

- 참조 - test-airflow/my-pod, search-airflow/Dockerfile
- 참조 - test-airflow/custom-values.yaml (원본 파일 - search-iac-airflow-v3/custom-values.yaml)
- 환경변수 읽어오기 적용 - AIRFLOW\_\_POD\_\_EXECUTION\_ENVIRONMENT, AIRFLOW\_\_TASK\_\_POD\_IMAGE - test-airflow repo

```
configmap-airflow-custom-env.yaml 적용
k apply -f configmap-airflow-custom-env.yaml

custom-values.yaml 작성 (원본 파일 custom-values.yaml.org)
extraEnvFrom: 작성하여 환경 변수 읽어오도록 적용

docker image build
$ cd my-pod
$ cat Dockerfile
FROM python:3.12-slim

USER root
RUN ln -sf /usr/share/zoneinfo/Asia/Seoul /etc/localtime

ARG UNAME=airflow
ARG UID=50000
ARG GID=50000
RUN groupadd -g $GID $UNAME
RUN useradd -m -u $UID -g $GID -s /bin/bash $UNAME
USER $UNAME
WORKDIR /home/${UNAME}

COPY ./requirements.txt .
RUN pip install --user -r requirements.txt

COPY ./tasks /opt/airflow/tasks

$ docker login dev.pcr.kr
$ docker build --platform linux/amd64 --pull --tag my-airflow-pod:0.0.1 .
$ docker tag my-airflow-pod:0.0.1 dev.pcr.kr/1001291/my-airflow-pod:0.0.1
$ docker push dev.pcr.kr/1001291/my-airflow-pod:0.0.1

Airflow upgrade
helm upgrade airflow apache-airflow/airflow --namespace airflow --debug --values custom-values.yaml
```

## Dag 수행

```
service account 생성
k apply -f worker-kubernetes-serviceaccount.yaml

custom-values.yaml 내용 추가

Airflow upgrade
helm upgrade airflow apache-airflow/airflow --namespace airflow --debug --values custom-values.yaml

airflow 웹 접근하여 tutorial 수행 (성공)
```

## 로컬환경 task 수행 방법

search-airflow repo 참조

- cd search-airflow
- python3.12 환경에서 requirements.txt 설치
  - pip install -r requirements.txt
- tasks 디렉토리 library 경로 추가
  - export PYTHONPATH=\$PYTHONPATH:~/Desktop/workspace-old/search-airflow
- cd tasks/sample
- dev-srch-common-role로 assume role 수행 - aws-assume-role 수행후 출력되는 export문 수행
- python conn\_es.py

```
(python3.12) 1001291@SKP1001291MN003 ~/Desktop/workspace-old/search-airflow/tasks/samples % python conn_es.py
{'name': '4ab0bc9731b0651f3fb08ec83c68ec5d', 'cluster_name': '175979101058:dev-search-es-219', 'cluster_uuid': 'M3pXmjm5RviQx4Ss5jRuNA',
'version': {'number': '7.10.2', 'build_type': 'tar', 'build_hash': 'unknown', 'build_date': '2026-03-27T13:02:49.865924982Z', 'build_snapshot': False,
'.lucene_version': '9.12.1', 'minimum_wire_compatibility_version': '7.10.0', 'minimum_index_compatibility_version': '7.0.0'}, 'tagline': 'The OpenSearch
Project: https://opensearch.org/'}
{'date_field': '20251231235959'}
```

## 참고 - Custom airflow image 생성, registry 등록

- Official airflow image 사용하면 됨. 추후 dag를 airflow image로 넣을 경우 Custom airflow image 사용 가능
- 참조 - <https://airflow.apache.org/docs/helm-chart/stable/quick-start.html>

```
Dockerfile 생성
search-airflow repo 아래 tasks 디렉토리 포함 image 생성 (dag 로딩 가능하게 하기 위해 tasks 포함) -> pod image만 tasks 포함시키면 되는지 체크
mkdir my-airflow && cd my-airflow
cp ~/search-airflow/tasks ./

cat <<EOM > Dockerfile
FROM apache/airflow
COPY . .
EOM

docker build & 등록 (dev.pcr.kr) - arm64 build
docker build --pull --tag my-airflow:0.0.1 .
docker login dev.pcr.kr
docker tag my-airflow:0.0.1 dev.pcr.kr/1001291/my-airflow:0.0.1
docker push dev.pcr.kr/1001291/my-airflow:0.0.1

docker build (linux) - amd64(linux) build - 맥북에서 build시 arm64 image가 build된 것으로 보임
platform 확인 방법 - Run uname -m on a node or a temporary pod to see if it is x86_64 (AMD64) or aarch64 (ARM64).
docker build --platform linux/amd64 --pull --tag my-airflow:0.0.2 .
docker tag my-airflow:0.0.1 dev.pcr.kr/1001291/my-airflow:0.0.2
docker push dev.pcr.kr/1001291/my-airflow:0.0.2

Airflow upgrade with custom airflow image
helm upgrade airflow apache-airflow/airflow --namespace airflow \
--set images.airflow.repository=dev.pcr.kr/1001291/my-airflow \
--set images.airflow.tag=0.0.2 --debug --values custom-values.yaml
```

## 참고 - pv 경로 변경시 적용 방법

```
airflow-dags pvc를 사용하는 statefulsets, deployments 조회
k -n airflow describe pvc airflow-dags
Name: airflow-dags
Namespace: airflow
StorageClass: manual05
Status: Bound
Volume: dags-pv
Labels: <none>
Annotations: pv.kubernetes.io/bind-completed: yes
 pv.kubernetes.io/bound-by-controller: yes
Finalizers: [kubernetes.io/pvc-protection]
Capacity: 5Gi
Access Modes: RWO
VolumeMode: Filesystem
Used By: airflow-dag-processor-59ccf9d769-brgt6
 airflow-triggerer-0
Events: <none>

statefulsets, deployments 삭제
k -n airflow delete statefulsets airflow-triggerer
k -n airflow delete deployments airflow-dag-processor

pvc, pv 삭제후 재생성
airflow upgrade 적용
helm upgrade airflow apache-airflow/airflow --namespace airflow --debug --values custom-values.yaml
```

## 참고 - 성수 k8s airflow → AWS OpenSearch 연동

- 로컬환경 task 수행시 assume role 수행하여 연동 성공하였으나 pod에서 권한 부여 방법 불확실
- OpenSearch IP 접근 권한 부여 테스트 - VPC 환경 OpenSearch의 경우 적용 안됨 (<https://docs.aws.amazon.com/opensearch-service/latest/developerguide/ac.html#ac-types-ip>)
- IAM user 생성 등을 통해 접근 부여하는 방식 가능
- OpenSearch와 연동하기 보다 성수 Elasticsearch 별도 구성하는 방안도 괜찮음

## 기타 참고

- ncloud nfs client provisioner - [90 Elasticsearch 추가 검증](#)
- k8s postgresql 설치
  - <https://velog.io/@manarc/Provisioning-PostgreSQL-standalone#storageclass>
    - storage class, pv 생성 → helm의 custom value 설정에서 storage class 지정 방식
- storage class manual vs no-provisioner

쿠버네티스(Kubernetes)에서 manual 스토리지 클래스 이름과 [kubernetes.io/no-provisioner](#) 프로비저너(Provisioner)는 **직접 수동으로 퍼시스턴트 볼륨(PV)을 생성·관리**해야 한다는 공통점이 있지만, 명칭의 정의와 사용 목적에서 차이가 있습니다. [1, 2]

핵심 차이 요약

| 구분    | manual (일반적인 사용자 정의 이름)                                      | <a href="#">kubernetes.io/no-provisioner</a> (내장 프로비저너) |
|-------|--------------------------------------------------------------|---------------------------------------------------------|
| 정의    | 관리자가 임의로 지은 스토리지 클래스의 <b>이름(Name)</b>                        | 동적 프로비저닝을 하지 않음을 명시하는 <b>프로비저너(Provisioner) 값</b>       |
| 역할    | 정적 프로비저닝(Static Provisioning) 환경에서 PV와 PVC를 매칭하기 위해 사용하는 식별자 | 쿠버네티스에 "이 스토리지 클래스는 자동 생성이 불가하니 수동 PV를 써라"고 알리는 기능      |
| 주요 용도 | 사용자가 임의로 지정한 이름 (storageClassName: manual)                   | 로컬 스토리지(local)나 호스트 경로 등 수동 바인딩이 필수인 리소스                |

상세 설명

### 1. manual 스토리지 클래스

- **개념:** manual은 사용자가 metadata: name: manual 형태로 직접 생성한 **StorageClass 객체의 이름**일 뿐입니다. [1]
- **동작 방식:** 관리자가 특정 백엔드 스토리지(예: hostPath 등)를 연결한 PersistentVolume(PV)을 미리 생성하고 storageClassName: manual로 지정해 둡니다. 이후 사용자가 storageClassName: manual을 명시한 PVC를 요청하면, 쿠버네티스는 이름이 일치하는 미리 만들어진 PV를 찾아 바인딩합니다. [1, 2]
- **특징:** 이름일 뿐이므로 내부 provisioner 필드에 다른 드라이버를 넣을 수도 있으며, 관습적으로 정적 할당 방식을 쓸 때 이름을 manual이나 static으로 흔히 지정합니다. [1, 2]

## 2. kubernetes.io/no-provisioner 스토리지 클래스

- **개념:** kubernetes.io/no-provisioner는 쿠버네티스 내부에서 제공하는 **특별한 프로비저너** 값입니다. [1]
- **동작 방식:** StorageClass의 provisioner 필드에 kubernetes.io/no-provisioner를 적어두면, PVC가 생성되어도 외부/내부 프로비저너가 백엔드 스토리지를 자동으로 생성(Dynamic Provisioning)하지 않습니다. [1, 2]
- **주요 용도:** 주로 노드의 로컬 디스크를 직접 사용하는 **Local Persistent Volume**이나 관리자가 하드코딩으로 관리해야 하는 정적 볼륨 구성 시 표준적으로 사용됩니다. [1]
- **특징:** 이 프로비저너를 사용하는 스토리지 클래스를 지정한 경우, 반드시 관리자가 직접 해당 조건(용량, 접근 모드, 노드 어피니티 등)에 맞는 PV를 수동으로 생성해 두어야 PVC가 정상적으로 바인딩됩니다. [1]

### 관계 및 실무 활용

실무에서는 두 개념을 혼용해서 함께 사용하기도 합니다. 즉, provisioner: kubernetes.io/no-provisioner로 설정된 스토리지 클래스의 이름을 manual이나 local-storage로 지정하여, 클러스터 사용자에게 "이 스토리지 영역은 자동 생성되지 않으며 수동으로 연결해야 한다"는 의도를 명확히 전달하는 방식으로 구성합니다. [1, 3]

## airflow aws ec2 설치

- <https://airflow.apache.org/docs/apache-airflow/stable/start.html>
- <https://code.skplanet.com/projects/SEAR/repos/search-iac-airflow-v3/browse>
- 설치 서버 - 성수센터 THYVMtx-rockylinux9-dev01
- 설치 및 기동
  - default python 버전 변경 필요
  - pip install "apache-airflow[celery]==3.2.2" --constraint "https://raw.githubusercontent.com/apache/airflow/constraints-3.2.2/constraints-3.12.txt"
    - constraints-3.12.txt를 직접 다운받아 서버로 복사후 아래 명령 수행
    - /usr/bin/pip-3.12 install "apache-airflow[celery]==3.2.2" --constraint "/constraints-3.12.txt"
  - airflow standalone
  - 패스워드 확인 - cat ~/airflow/simple\_auth\_manager\_passwords.json.generated
- dag run - airflow dags test tutorial 2026-08-18
- dag example unload
  - airflow.cfg 설정 파일 편집 - load\_examples = False
  - airflow db reset 수행
- Airflow restart
  - airflow api-server -D
  - airflow scheduler -D
- Run dag, task

```
Run dag
python3.12 ~/airflow/dags/tutorial.py
airflow dags test tutorial_es_connect 2026-08-18

Run task
command layout: command subcommand [dag_id] [task_id] [(optional) date]
testing print_date
airflow tasks test tutorial print_date 2015-06-01

testing sleep
airflow tasks test tutorial sleep 2015-06-01
```

- 필요 library 설치 - search-airflow repo 참조 - requirements.txt

```
$ cat requirements.txt
PyMySQL==1.1.1
SQLAlchemy==2.0.32
boto3==1.35.2
redis==5.0.8
requests==2.32.3
coverage==7.6.1
pytest==8.3.2
pytest-cov==5.0.0
pytest-mock==3.14.0
moto==5.0.13
pylint==3.2.6
opensearch-py==2.8.0
ImportError: cannot import name 'soft_unicode' from 'markupsafe' issue
MarkupSafe==2.1.5
kubernetes==30.1.0
python-dateutil==2.9.0.post0
pytz==2024.1
botocore==1.35.2
```

- dag ~/airflow/dags/tutorial\_base.py 작성 (PythonOperator 사용)
  - tutorial.py - KubernetesPodOperator 사용

```

-*- coding: utf-8 -*-
import os
import runpy
import sys

import pendulum
from airflow.providers.standard.operators.python import PythonOperator
#from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
from airflow.sdk import DAG

default_args = {
 'owner': 'airflow',
 'depends_on_past': True,
 'email': ['chszard@sk.com'],
 'email_on_failure': False,
 'email_on_retry': False,
 'retries': 1,
 'retry_delay': timedelta(minutes=1),
}

airflow_home = "/app/puser/airflow"
execution_env = 'dev'

os.environ["PYTHONPATH"] = airflow_home
os.environ["APP_ENV"] = execution_env
print(sys.path)
sys.path.append(airflow_home)
print(sys.path)

def run_script(script_path):
 runpy.run_path(script_path, run_name="__main__")

with DAG('tutorial_base', default_args=default_args, description='python operator run',
 start_date=pendulum.datetime(2026, 6, 1, tz="Asia/Seoul"),
 schedule=None) as dag:
 task1 = PythonOperator(
 task_id="connect_es",
 python_callable=run_script,
 op_kwargs={
 "script_path": "/app/puser/airflow/tasks/samples/print_task_01.py",
 },
)

task1

```

- sys.path 설정 - tasks library 사용을 위해 필요
- 필요 library 설치 - pip3.12 install -r requirements.txt
- OpenSearch 연동을 위한 assume role 수행 - credential 설정해야 ES 연동 가능 (성공)
- 성수센터 → AWS 방화벽 문제로 연결 불가
- 결론 - 방화벽 오픈후 재테스트 or 성수센터에 ES 구축후 테스트